AI-Powered Full Stack Developer Program

HOMEWORK SHEET

Week 2 · Day 1

How the Web Works + HTML Core

Homework HW1–HW4: tasks, time boxes, submit checklist, self-check and interview prep

Momo House · TSI Canteen · Aarav · Sunita · Didi

Stage 1, AI as Tutor: AI may explain; you type every line.

Tech Skills Institute (TSI) Nepal · Kathmandu

Week 2 · Day 1 Homework: How the Web Works + HTML Core

File: homework/W2D1-Homework-Sheet.md

TSI Nepal · AI-Powered Full Stack Developer Program · Week 2, Day 1 · Layer: Frontend

Tonight in one sentence: you repair a broken site, build one new page, then test yourself, so tomorrow you can dress the skeleton with CSS.

Core time: about 2.5 hours. **HW1 and HW3 are the must-do (1 hour).** Do HW2 if you can (about 1.5 hours). **Stretch:** HW4 adds about 30 minutes. It is optional.

Before you start (5 minutes)

1. Open VS Code. **File → Open Folder → `W2D1`.** Open the folder, not a single file.
2. Check that **Live Server** and **HTMLHint** are installed (Extensions panel, `Ctrl+Shift+X`). If not, install them now.
3. Open the **Problems** panel with `Ctrl+Shift+M`. Keep it open all evening. It is your spell-checker for HTML.
4. Keep `notes/11-day1-cheatsheet.md` open in a second tab.

The browser is a polite guesser. It never shows you an error. It guesses and carries on. Your tools (HTMLHint, F12 Elements, the address bar) are how you catch its guesses.

The plan for tonight

#	Id	Title	Type	Time	Folder
1	**D1-HW1**	Kathmandu Treks repair	bug-hunt, 12 bugs	40 min (must-do)	`homework/D1-HW1-kathmandu-treks-repair/`
2	**D1-HW2**	Momo House catering page	build	about 1.5 h (if you can)	`homework/D1-HW2-momo-house-catering-page/`
3	**D1-HW3**	Self-check & interview prep	questions	20 min (must-do)	`homework/D1-HW3-Self-Check-and-Interview-Prep.md`
4	**D1-HW4**	Thamel Guesthouse replay (**stretch**)	bug-hunt, 12 bugs	+30 min	`homework/D1-HW4-thamel-guesthouse-replay/`

Do them in this order. Short on time? Do HW1 and HW3 first; they are the must-do. HW1 warms you up on every bug type from today. HW2 is the real build. HW3 checks that you can *explain* what you did. HW4 is for anyone who wants more.

D1-HW1 · Kathmandu Treks repair (40 min, must-do)

The story. A small trekking company in Thamel has a 3-page website: a home page, a treks page with a price table, and a booking form. Their nephew built it in one night. It has **12 bugs**. You are the repair crew.

Files: `index.html`, `treks.html`, `booking.html`, and `images/` (Everest, Annapurna, Langtang).

What you practise: every bug type from today, on a site you have not seen before.

- skeleton and `<head>` mistakes
- file names and paths (including capital letters)
- links between pages and `#id` jumps
- lists inside lists

- tables: header cells, `rowspan`
- forms: labels, input types, radio groups, buttons

How to work:

1. Read the folder's `README.md` first. It has a **"fix this first"** note. Read it.
2. Open each page with Live Server. Click every link. Look at every picture.
3. Read the Problems panel (`Ctrl+Shift+M`).
4. For anything that looks fine but feels wrong, open **F12 → Elements** and compare what the browser built with what you wrote.
5. Fill the form and press the button. Read the address bar.

Done when:

- ☐ All 3 pages open, every image shows, and every link goes to the right place.
- ☐ The Problems panel shows **0 problems** for all 3 files.
- ☐ You wrote down every bug you found: file, what was wrong, how you saw it (V, D(HH), D(F12), D(URL), H or R).

Scoreboard: `I found __ / 12. The bug that fooled me longest: _____`

D1-HW2 · Momo House catering page (about 1.5 h, if you can)

The story. Didi's Momo House now cooks for offices and parties. She needs a new page, `catering.html`, so customers can see the packages and send a booking request.

Your starter (in the HW2 folder) is the repaired Momo House site from class (D1-15): `index.html`, `menu.html`, `order.html`, `about.html` and the 4 Momo House pictures.

Build `catering.html` with:

1. The full skeleton: `<!DOCTYPE html>`, `lang="en"`, `meta charset`, `meta viewport`, a real `<title>`.
2. The same nav as the other pages, plus a new **Catering** link.
3. One `<h1>`, then headings in order (no jumps).
4. **One table of packages**: a `caption`, a header row with `th scope="col"`, at least 3 packages (for example Office Lunch, Birthday Party, Wedding Feast), with plates and price in Rs.
5. **One form** with `method="get"`:
   - a `date` field (event date)
   - a `number` field (how many people), with `min`
   - a **package** radio group (one shared `name`, a `value` on each radio) inside `<fieldset>` + `<legend>`
   - a `textarea` for notes (with a proper closing tag)
   - a submit button **inside** the form
   - a `<label>` for every input, with `for` matching the `id`
6. Add the **Catering** link to the nav on **all 5 pages**.

Test it like the class did:

- Click every label. The cursor must jump into its box.
- Fill the form and submit. Read the address bar. Every answer must appear as `name=value`.

Done when:

- ☐ 5 pages, and the nav on every page has 5 links that all work.
- ☐ The table has a caption, header cells and at least 3 packages.
- ☐ Submitting the form shows every field in the address bar, for example `catering.html?event-date=2026-12-20&people=60&package=wedding&notes=…`.

☐ The Problems panel shows 0 problems for all 5 files.

Bonus: add a `tfoot` row to the table ("Prices include Rs. 0 delivery inside Ring Road"), and a "back to top" link that jumps to `#top`.

D1-HW3 · Self-check & interview prep (20 min, must-do)

Open `homework/D1-HW3-Self-Check-and-Interview-Prep.md`.

1. **Part 1, self-check (10 questions):** answer from memory first. Then check your notes. Mark each one ✔ (knew it), ~ (half) or ✗ (didn't).
2. **Part 2, interview questions (12 questions):** say each answer **out loud**, as if a Kathmandu tech lead asked you. 20–40 seconds each. Interviews are spoken.
3. Your instructor shares the answer key after you try. Compare. Re-read the notes for every ✗.

Done when: you have an answer (written or spoken) for all 22 questions, and a list of your ✗ topics.

D1-HW4 · Thamel Guesthouse replay (stretch, +30 min)

Optional. Do it only after HW1–HW3.

The story. A guesthouse in Thamel has a home page and a booking page. It has **12 bugs**. They are the bugs we did not have time for in class, so some of them are new to you.

Files: `index.html`, `booking.html`, `images/` (guesthouse, deluxe room).

What you practise: the head and the title, image names and `alt`, table rows and `caption`, and the trickier form bugs (`select`, `option`, `required`, missing `name`).

Hint: some bugs only show up when you **submit** the form, or when you **leave a field empty** and submit. Some only show up in **F12 → Elements**. One only breaks on a real Linux server.

Done when: both pages pass HTMLHint, the room image shows, and an empty or wrong form cannot be sent.

Scoreboard: `I found __ / 12. The new thing I learnt: _____`

Submit checklist (before the next class)

Tick every box. Zip each homework folder and send it the way your instructor asks. Also bring your `W2D1` folder on your laptop.

- ☐ **HW1:** the 3 Kathmandu Treks pages are fixed, with 0 problems in the Problems panel.
- ☐ **HW1:** my bug list (file + bug + visibility tag) is in `homework/D1-HW1-kathmandu-treks-repair/my-bugs.md`.
- ☐ **HW2:** `homework/D1-HW2-momo-house-catering-page/catering.html` exists, and all 5 pages link to it.
- ☐ **HW2:** I submitted the form once and copied the address bar into `homework/D1-HW2-momo-house-catering-page/my-url.txt`.
- ☐ **HW3:** I answered all 22 questions and wrote my ✗ list at the bottom of the HW3 file.
- ☐ *(Stretch)* **HW4:** the Thamel Guesthouse pages are fixed.
- ☐ I typed every line myself. I can explain every line. (Ask yourself: "Could I explain this line to Aarav?")
- ☐ **One volunteer** will show their HW2 page on the projector at the start of Day 2. It could be you.

AI rule for this week

Stage 1, AI as Tutor. AI may **explain** a concept or an error to you. **You type every line.** Try first, then ask.

A good prompt for tonight:

> "My HTML table has one cell sticking out on the right of the second row. I used `rowspan="2"` in the first row. Explain how rowspan changes how many cells the next row needs. Don't write the code for me."

A bad prompt: "Fix my booking.html." (You learn nothing, and you can't explain the result.)

Stuck?

1. Re-read the folder's `README.md` hints.
2. Open the matching notes file: paths → `notes/06-links-and-paths.md`, images → `07`, tables → `08`, forms → `09`, "it looks fine but HTMLHint is angry" → `10`.
3. Open **F12 → Elements**. Compare what the browser built with what you wrote.
4. Ask a classmate to *point* (not type).

Remember the One Rule: you may only ship code you can explain, line by line. Next class (Day 2): semantic HTML and accessibility, then the skeleton gets its outfit: CSS.

D1-HW1 · Kathmandu Treks repair

File: homework/D1-HW1-kathmandu-treks-repair/README.md

Time at home: about 40 min (must-do) · **Type:** 🐞 Bug-hunt · **Work:** alone

Goal: Repeat every bug type from today on a new 3-page site (spaced practice).

Open it

Open the `W2D1` folder in VS Code. Open this folder's `index.html` with Live Server.

Your task

Kathmandu Treks is a small trekking company in Thamel. Their website has 12 bugs across 3 pages.

```
D1-HW1-kathmandu-treks-repair/
├── index.html      4 bugs
├── treks.html      3 bugs
├── booking.html    5 bugs
└── images/         (everest.svg, annapurna.svg, langtang.svg)
```

There are 12 bugs. Find at least 10.

> ⚠ **Fix this first:** `index.html` is completely blank. That is on purpose. If a page is completely blank, fix this first.

For each bug, write one line in a file called `my-bugs.md` in this folder: `page · line · what was wrong · how I saw it (V / D(HH) / D(F12) / D(URL) / D(W3C) / H / R)`

Hints (one per bug type)

- **Blank page**: some tags swallow everything after them if they are not closed.
- **File names**: compare every `src` with the real file name, letter by letter, capitals too.
- **Links**: every page link ends in `.html`; every `#name` must match an `id`.
- **Nested lists**: the sub-list lives inside its parent `<li>`.
- **Tables**: header cells; count the cells when a `rowspan` is used.
- **Forms**: labels, input types (phone, date), radio names, and what the button does.

How bugs hide (the visibility legend)

Tag	Meaning	How you see it
V	You can **see** it on the page	your eyes
D(HH)	HTMLHint underlines it in VS Code (Problems panel: Ctrl+Shift+M)	HTMLHint
D(F12)	Only the Elements panel shows what the browser really built	F12 → Elements
D(URL)	Submit the form and read the address bar	the URL bar
D(W3C)	Only the W3C validator reports it	validator.w3.org (Wi-Fi)
H	Hidden on Windows; breaks on a real Linux server (capital letters in a file name)	the instructor explains
R	Rule check: no tool shows it. Read the code and check the rule	your brain + the notes

Done looks like

- All 3 pages show. Every link works.
- The tables line up. The "Go straight to the booking form" link lands on the form.
- The form: labels work, phone has no arrows, the date has a calendar, only one trek can be ticked, and **Book now** sends the form (read the URL).

- `my-bugs.md` has 12 lines (or at least 10).

Scoreboard

```
I found __ / 12. The bug I am proudest of: ________
```

Stuck?

1. Blank page? Look at the `<head>` first. Which tag there has text inside it?
2. One bug is invisible on Windows. Read every file name in `images/` and compare.
3. Compare with the class bug-hunts: D1-04, D1-07, D1-09, D1-11, D1-13.

Submit

Zip this folder (with `my-bugs.md` inside) and send it the way your instructor asks. See `homework/W2D1-Homework-Sheet.md` for the checklist.

D1-HW2 · Momo House catering page

File: homework/D1-HW2-momo-house-catering-page/README.md

Time at home: about 1.5 h (if you can; HW1 and HW3 come first) · **Type:** 🔨 Build · **Work:** alone

Goal: Add a new page to a real site: one table and one form, linked from every page.

Open it

Open the `W2D1` folder in VS Code. Open this folder's `index.html` with Live Server. This is the **repaired** Momo House site from class (D1-15). Everything works.

Your task

Didi wants to offer **catering** (momo for weddings, birthdays and office parties). Build `catering.html`.

1. Create `catering.html` next to `index.html`. Start with `!` + Tab. Copy the logo and nav from `about.html`.
2. Add an `<h1>` and one short intro paragraph.
3. Add **one table** of packages:
   - ☐ `<caption>`, `<thead>` with `<th scope="col">`, `<tbody>`
   - ☐ 3 packages (for example Small / Party / Wedding), with what you get and the price per person
   - ☐ Bonus: a `<tfoot>` row with `colspan` ("Minimum 20 people …")
4. Add **one form** (`method="get"`):
   - ☐ the event **date** (`type="date"`)
   - ☐ the **number of people** (`type="number"`, with `min`)
   - ☐ the **package**: a radio group with one shared `name` and a `value` on each, inside `<fieldset>` + `<legend>`
   - ☐ **notes**: a `<textarea>` (with a real `</textarea>`!)
   - ☐ a submit button
   - ☐ every input has a `<label for="…">`
5. Add a **Catering** link to the nav on **all 5 pages** (index, menu, order, catering, about).
6. Check: the Problems panel (Ctrl+Shift+M) shows **0** problems on all 5 pages.

Done looks like

- From any page you can reach any other page in one click.
- The table has a caption and a bold header row.
- Fill the form and submit. The URL shows something like: `catering.html?event-date=2026-12-20&people=60&package=wedding&notes=…`
- HTMLHint: 0 problems on all 5 pages.

Stuck?

1. Copy, don't retype: the order form in `order.html` already has most of the parts you need.
2. The nav is the same 5 links on every page. Make it once, then copy it into the other 4 pages.
3. Look at `notes/08-tables.md` and `notes/09-forms.md`.

Submit

Zip this folder and send it the way your instructor asks. See `homework/W2D1-Homework-Sheet.md` for the checklist.

D1-HW3 · Self-Check & Interview Prep: How the Web Works + HTML Core

File: homework/D1-HW3-Self-Check-and-Interview-Prep.md

TSI Nepal · AI-Powered Full Stack Developer Program · Week 2, Day 1 · Layer: Frontend · about 20 minutes

Mode: Theory + coding, because Kathmandu interviewers ask HTML viva-style ("What is…? Difference between…?") and then ask you to mark up or fix a small page.

How to use this sheet

1. **Part 1** checks your memory. Answer on paper or in this file, without the notes. Then mark each one ✔ (knew it), ~ (half) or ✗ (didn't).
2. **Part 2** is an interview. Stand up, and say each answer **out loud** in 20–40 seconds. Interviews are spoken, not written.
3. Your instructor shares the answer key after you try. Don't peek first: a question you struggle with is the one you remember.

How you'll be asked about this

- **Written / MCQ test:** "Which tag makes the biggest heading?", "Which attribute opens a link in a new tab?", "What does `<` print?"
- **Viva round:** "What is HTML?", "What is the difference between `<strong>` and `<b>`?", "What is DOCTYPE for?", "Absolute vs relative path?"
- **Machine test:** "Make this signup form" or "Fix this page" in 20–30 minutes. They check labels, input types, and that the page is valid.
- **What they are really checking:** can you *explain* why, not only recite. "The browser guesses when you are wrong" is a stronger answer than a definition from a textbook.

Topic map

Sub-topic	Why interviewers care	Questions
How the web works	Every web job starts with request → response	S1, T1
Skeleton, head/body, DOCTYPE	Shows you know what every page needs	S2, S3, T2, C1
Elements, tags, attributes, nesting	The vocabulary of HTML	S4, T3
Text tags and entities	Meaning vs looks	S5, T4
Lists	Picking the right structure	S6
Links and paths	The #1 bug on real sites	S7, T5
Images and alt	Accessibility and broken pictures	S8, T6
Tables	Data, not layout	S9, C2
Forms	Every app collects data	S10, T7, C3, C4
Debugging / validation	How juniors save time	T8

Part 1: Self-check (10 questions)

S1. In the postal-system picture of the web, what are the **DNS**, the **IP address** and the **response**? Give one line for each.

S2. Write, from memory, the full skeleton of an HTML page with the title "Momo House". (10 lines.)

S3. What goes in the `<head>` and what goes in the `<body>`? Use the parcel picture.

S4. In `<a href="menu.html">Menu</a>`, which part is the **element**, which is the **tag**, which is the **attribute**, and which is the **value**?

S5. You want the browser to print the text `<p>` on the page, not start a paragraph. What do you type?

S6. Didi writes her shopping list (cabbage, flour, chicken) and her recipe steps (mix, fold, steam). Which list tag does each one get, and why?

S7. `index.html` is in the `W2D1` folder. `about.html` is in `W2D1/pages/`. Write the link **from `about.html` back to `index.html`**.

S8. Why is `<img>` called "a sticker, not an envelope"? And what is `alt` for?

S9. A table row has `rowspan="2"` on its first cell. The table has 3 columns. How many `<td>` must the **next** row have?

S10. Two radio buttons for "Morning" and "Evening" batch. Both can be ticked at the same time. What is wrong, and what does the URL show if they also have no `value`?

At the bottom of this file, write: `My ✗ topics:` _____

Part 2: Interview questions (12 questions)

Theory (say it out loud)

T1. (Warm-up) What happens when you type `momohouse.com.np` in the browser and press Enter? Keep it to 5 steps.

T2. (Warm-up) What is `<!DOCTYPE html>` for? What happens if you forget it?

T3. (Warm-up) What is the difference between an **element**, a **tag** and an **attribute**?

T4. (Core) What is the difference between `<strong>` and `<b>`? And `<em>` and `<i>`? Which one should you use, and why?

T5. (Core) What is the difference between an **absolute** path and a **relative** path? Give one example of each, and say why `C:\Users\Aarav\Desktop\site\index.html` is a bad link.

T6. (Core) Why should every `<img>` have an `alt`? When is `alt=""` (empty) the right answer?

T7. (Core) In a form, what do the `name`, `id` and `for` attributes each do? Which one does the server see?

T8. (Stretch) "The browser never shows an HTML error." Is that true? How do you find HTML mistakes, then? Name three tools.

Code (write it or say it)

C1. Find the bugs. This page looks almost fine in Chrome. Name **three** problems.

✗ Broken on purpose: find what is wrong

```
<html>
<head>
  <titel>Aarav's Page</titel>
</head>
<body>
  <h1>Hello Kathmandu
  <p>I live in <strong>Kathmandu</p></strong>
</body>
</html>
```

C2. Predict. How many columns does the browser draw for this table, and where does the text "Tea break" end up?

✗ *Broken on purpose: find what is wrong*

```
<table>
  <tr><th>Time</th><th>Sun</th><th>Mon</th></tr>
  Tea break
  <tr><td>10:00</td><td colspan="2">HTML lab</td></tr>
</table>
```

C3. Write it (5 minutes). Write a form for the TSI Canteen: a labeled **email** field, a labeled **phone** field (right type!), a "Veg / Non-veg" radio group, and a submit button. Use `method="get"`.

C4. Find the bug. Sunita says: "I click Submit and nothing happens." Why?

✗ *Broken on purpose: find what is wrong*

```
<form method="get">
  <label for="msg">Message</label>
  <textarea id="msg" name="msg" />
</form>
<button type="submit">Send</button>
```

Talk about your project

Be ready for these in a "walk me through it" round:

- "In the Momo House repair (D1-15), which bug was hardest to find? How did you find it?"
- "In your catering page (D1-HW2), how did you check that the form sends every field?"

Remember the One Rule: you may only ship code you can explain, line by line.

My ✗ topics: _____

D1-HW4 · Thamel Guesthouse replay (stretch, optional)

File: homework/D1-HW4-thamel-guesthouse-replay/README.md

Time at home: about 30 min · **Type:** Bug-hunt · **Work:** alone

Goal: The last of the bugs that were cut from the class hunts. A replay of the whole day.

Open it

Open the `W2D1` folder in VS Code. Open this folder's `index.html` with Live Server.

Your task

Thamel Guesthouse has a two-page website. It *looks* nearly fine. Many of these bugs only show with a tool, a submit, or a careful read.

```
D1-HW4-thamel-guesthouse-replay/
├── index.html      4 bugs
├── booking.html    8 bugs
└── images/         (guesthouse.svg, room-deluxe.svg)
```

There are 12 bugs. Find at least 10.

For each bug, write one line in a file called `my-bugs.md` in this folder: `page · line · what was wrong · how I saw it (V / D(HH) / D(F12) / D(URL) / D(W3C) / H / R)`

Hints (one per bug type)

- **Skeleton**: the first line, and where `<title>` lives.
- **Pictures**: `alt`, and capital letters in file names.
- **Tables**: every row has `<tr>`; the caption comes first.
- **Forms**: input types, spelling of attributes, `name` on every field, options inside the select, radio names, and where the button is.
- **Use every tool**: Problems panel, F12 → Elements, submit + read the URL, and (Wi-Fi) the W3C validator.

How bugs hide (the visibility legend)

Tag	Meaning	How you see it
V	You can **see** it on the page	your eyes
D(HH)	HTMLHint underlines it in VS Code (Problems panel: Ctrl+Shift+M)	HTMLHint
D(F12)	Only the Elements panel shows what the browser really built	F12 → Elements
D(URL)	Submit the form and read the address bar	the URL bar
D(W3C)	Only the W3C validator reports it	validator.w3.org (Wi-Fi)
H	Hidden on Windows; breaks on a real Linux server (capital letters in a file name)	the instructor explains
R	Rule check: no tool shows it. Read the code and check the rule	your brain + the notes

Done looks like

- Problems panel: 0 on both pages.
- The form stops an empty name and a bad email. Deluxe is in the drop-down. Only one room type can be ticked.
- Submit and read the URL: it has `guest`, `email`, `room` and `room-type`.
- `my-bugs.md` has 12 lines.

Scoreboard

```
I found __ / 12. The bug I am proudest of: ________
```

Stuck?

1. The tab shows the title, so the title is fine? Look again: *where* is it?
2. Paste each page into validator.w3.org (Wi-Fi). It finds the one bug no other tool shows.
3. Every bug here appeared in class (D1-04, D1-09, D1-11, D1-13, D1-15).

Submit

Zip this folder (with `my-bugs.md` inside) and send it the way your instructor asks. See `homework/W2D1-Homework-Sheet.md` for the checklist.